

Python Programming

Assignment 1

Name: Pratiksha Purkar

Roll Number: 19

Jupyter Notebook and source code

[GitHub Repository](#)

0.1 Variables

What is it?

A variable is a container that used to store a value in memory.

Why do we need it?

Variables save information and reuse or change it later. They make programs easier to read than programs that repeat values everywhere.

Where and how is it used?

Variables are used in calculations, user input, conditions, loops, and almost every other part of a program. The assignment operator = stores a value.

```
student_name = "Alice"  
age = 22  
next_year_age = age + 1
```

0.2 Basic Data Types

Data types tell Python what kind of value is stored and what operations can be performed on it.

0.2.1 Integer (int)

What is it?

An integer is a whole number without a decimal point. Example :: 10, -4, or 0.

Why do we need it?

It is needed for counting and for calculations where fractional values are not required.

Where and how is it used?

Integers are used for age, marks, quantities, indexes, and loop counters.

```
number_of_students = 30
```

0.2.2 Float (float)

What is it?

A float is a number containing a decimal point. Example:: 3.14 or -2.5.

Why do we need it?

It is needed when a value may contain a fractional part.

Where and how is it used?

Floats are used for height, weight, temperature, averages, and scientific calculations.

```
temperature = 36.5
```

0.2.3 Boolean (bool)

What is it?

A Boolean has only two possible values: `True` or `False`.

Why do we need it?

It helps a program make decisions by representing whether a condition is true or not.

Where and how is it used?

Booleans are used in conditions, comparisons, loops, and status checks.

```
is_logged_in = True
```

0.2.4 String (str)

What is it?

A string is a sequence of characters written inside single, double, or triple quotation marks.

Why do we need it?

to store text information.

Where and how is it used?

Strings are used for names, messages, addresses, and other textual information.

```
course = "Scientific Computing"
```

0.3 Arithmetic Operators

What is it?

Arithmetic operators perform mathematical calculations. The main operators are `+`, `-`, `*`, `/`, `//`, `%`, and `**`.

Why do we need it?

They are needed to calculate totals, differences, products, averages, remainders, and powers.

Where and how is it used?

They are used between numeric values or variables.

Operator	Meaning	Example result
<code>+</code>	Addition	<code>10 + 3 = 13</code>
<code>-</code>	Subtraction	<code>10 - 3 = 7</code>
<code>*</code>	Multiplication	<code>10 * 3 = 30</code>
<code>/</code>	Division	<code>10 / 3 = 3.333...</code>
<code>//</code>	Floor division	<code>10 // 3 = 3</code>
<code>%</code>	Remainder	<code>10 % 3 = 1</code>
<code>**</code>	Power	<code>10 ** 3 = 1000</code>

0.4 Comparison Operators

What is it?

Comparison operators compare two values and produce `True` or `False`. They include `==`, `!=`, `>`, `<`, `>=`, and `<=`.

Why do we need it?

They allow programs to test relationships between values and make decisions.

Where and how is it used?

They are commonly used in `if` statements and loops.

```
marks = 70
print(marks >= 40)  # True
print(marks == 100) # False
```

0.5 Assignment Operators

What is it?

Assignment operators store or update values in variables. The simple operator is `=`. Short forms include `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, and `**=`.

Why do we need it?

They make code shorter and easier to read.

Where and how is it used?

They are used when totals, counters, or other values must be updated.

```
score = 10
score += 5  # Same as: score = score + 5
print(score) # 15
```

0.6 Type Conversion

What is it?

Type conversion changes one data type into another. `int()`, `float()`, `str()`, and `bool()`.

Why do we need it?

User input is always a string. Sometimes we need numbers for calculations. For example, `input()` returns text even when the user types a number.

Where and how is it used?

Convert the value before using it in a calculation or another type-specific operation.

```
age_text = "22"
age_number = int(age_text)
print(age_number + 1) # 23
```

0.7 Input and Output

0.7.1 The `input()` Function

What is it?

`input()` takes data from the user.

Why do we need it?

To allow users to enter information ,and allows it to work with information supplied at run time.

Where and how is it used?

Store the returned value in a variable. Convert it when numeric input is required.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

0.7.2 The print() Function**What is it?**

print() displays text or values on the screen.

Why do we need it?

It is used to show results, messages, and helpful information while testing a program.

Where and how is it used?

Pass one or more values inside its parentheses. An f-string can combine labels and variable values clearly.

```
print(f"Hello {name}. You are {age} years old.")
```

1 Strings**1.1 String Creation****What is it?**

A string is created by placing text inside quotes.

Why do we need it?

Different quote styles make it easier to include apostrophes, quotation marks, or multi-line text.

Where and how is it used?

Single and double quotes create normal strings. Triple quotes can create multi-line strings.

```
first = 'Python'
message = "Python is easy to learn!"
paragraph = """This text can
continue on another line."""
```

1.2 String Indexing**What is it?**

Indexing accesses one character by its position. The first character has index 0. Negative indexes count from the end, so -1 means the last character.

Why do we need it?

It is useful when a program needs a particular character.

Where and how is it used?

Write the index inside square brackets after the string.

```
word = "Python"
print(word[0])    # P
print(word[-1])   # n
```

1.3 String Slicing

What is it?

Slicing extracts part of a string using `string[start:stop:step]`. The stop position is not included.

Why do we need it?

It is useful for selecting, skipping, or reversing characters.

Where and how is it used?

Any part of the slice may be omitted. For example, `word[::-1]` reverses the string.

```
word = "Python"
print(word[0:3])    # Pyt
print(word[2:])     # thon
print(word[::-1])   # nohtyP
```

1.4 String Operations

What is it?

String operations combine, repeat, inspect, or change text. Useful operations include `+`, `*`, `len()`, `upper()`, `lower()`, `strip()`, `replace()`, and `split()`.

Why do we need it?

They help clean, format, search, and prepare text for display or processing.

Where and how is it used?

The `+` operator joins strings, `*` repeats them, and string methods return a new string. Strings themselves cannot be changed in place.

```
first = "Python"
print(first + " Programming")
print(first.upper())
print(len(first))
```

2 Lists

2.1 Creating Lists

What is it?

A list is an ordered collection written inside square brackets. It may contain values of the same or different types.

Why do we need it?

Lists let us keep many related values in one variable.

Where and how is it used?

Separate list items with commas.

```
fruits = ["Apple", "Banana", "Mango"]
mixed = [10, "Python", True]
```

2.2 List Indexing and Slicing

What is it?

Indexing selects one element, while slicing selects a range of elements.

Why do we need it?

They give direct access to the required part of a list.

Where and how is it used?

Lists use the same zero-based and negative indexing rules as strings.

```
fruits = ["Apple", "Banana", "Mango", "Orange"]
print(fruits[0])      # Apple
print(fruits[-1])     # Orange
print(fruits[1:3])    # ['Banana', 'Mango']
```

2.3 Updating Elements

What is it?

A list element can be replaced by assigning a new value to its index.

Why do we need it?

It allows stored information to be corrected or updated without creating a completely new list.

Where and how is it used?

Select an index on the left side of the assignment operator.

```
numbers = [10, 20, 30]
numbers[1] = 200
```

2.4 Nested Lists

What is it?

A nested list is a list that contains one or more lists as its elements.

Why do we need it?

It is useful for tables, matrices, grouped records, and other structured data.

Where and how is it used?

Use one index to select the inner list and another to select an item inside it.

```
matrix = [[1, 2], [3, 4]]
print(matrix[1][0]) # 3
```

2.5 Basic List Methods and Functions

Python provides list methods for modifying lists and built-in functions::

<code>append():</code>	Adds an item at the end. Example: <code>number.append(5)</code>
<code>insert(i, x):</code>	Adds an item at a specific position. Example: <code>number.insert(1, 5)</code>
<code>remove():</code>	Removes the first element.. Example: <code>number.remove(5)</code>
<code>pop(i):</code>	Removes and returns the item at index <code>i</code> . If the index is omitted, it removes the last item. Example: <code>number.pop()</code>
<code>sort():</code>	Arranges the list in ascending order by default. Example: <code>number.sort()</code>
<code>reverse():</code>	Reverses the current order of the list. Example: <code>number.reverse()</code>
<code>len(list):</code>	Returns the number of elements in the list. Example: <code>len(number)</code>

```
numbers = [30, 10, 20]
numbers.append(40)      # [30, 10, 20, 40]
numbers.insert(1, 15)   # [30, 15, 10, 20, 40]
numbers.remove(10)      # [30, 15, 20, 40]
last = numbers.pop()    # last = 40
numbers.sort()          # [15, 20, 30]
numbers.reverse()       # [30, 20, 15]
print(len(numbers))     # 3
```